

The MERN Stack Roadmap

A start-to-finish path through MongoDB, Express, React & Node — taught entirely through mini projects

This guide replaces long tutorials with 13 phases of small, buildable projects. Each phase targets the core skills you need before the next one makes sense — by the end you will have built (and can show) roughly 35 projects, finishing with 3 full capstones. Build every project yourself before moving on; don't just read the list.

Phase	Focus	You'll be able to...
0	JS & Git Warm-up	Write modern JS confidently and use Git/GitHub
1	React Fundamentals	Build interactive UIs with components, hooks & routing
2	Node.js Core	Understand what a runtime/server does under the hood
3	Express & REST APIs	Build and structure a real backend API
4	MongoDB & Mongoose	Model, query and relate data in a NoSQL database
5	Full-Stack CRUD	Connect a React frontend to your own Express+Mongo API
6	Auth & Security	Add signup/login, JWTs, hashed passwords, protected routes
7	State Management	Manage global state with Context and Redux Toolkit
8	Advanced React & Perf	Optimize renders, build custom hooks, handle scale
9	Real-Time Features	Add live updates with WebSockets / Socket.IO
10	Testing	Test APIs and components so bugs get caught early
11	Deployment & Docker	Ship a MERN app live and containerize it
12	Capstones	Combine everything into 3 portfolio-ready full apps

PHASE 0 — JavaScript & Git Warm-up

Skip this phase if you're already comfortable with ES6+ JS and basic Git — otherwise do it first.

1. JS Fundamentals Playground

Level: Beginner | **Core skills:** ES6+, array/object methods, async basics

- Practice destructuring, spread/rest, template literals, and arrow functions in small snippets
- Solve 10 short exercises using map/filter/reduce on arrays of objects
- Write 3 functions using async/await and try/catch around a fake fetch (setTimeout-based)

2. Vanilla JS To-Do List

Level: Beginner | **Core skills:** DOM manipulation, events, localStorage

- Add, complete, and delete tasks using plain JS and the DOM API (no framework)
- Persist the list in localStorage so it survives a page refresh
- Filter tasks by All / Active / Completed

3. Git & GitHub Practice Repo

Level: Beginner | **Core skills:** version control, branching, pull requests

- Initialize a repo, make commits with meaningful messages, and push to GitHub
- Create a feature branch, make changes, and open a pull request into main
- Practice resolving a merge conflict on purpose

PHASE 1 — React Fundamentals

Learn components, hooks, and routing before touching the backend at all.

1. Counter & Toggle App

Level: Beginner | **Core skills:** `useState`, event handlers, conditional rendering

- A counter with increment/decrement/reset buttons
- A show/hide toggle for a block of text
- Disable the decrement button once the counter hits 0

2. Digital Clock & Stopwatch

Level: Beginner | **Core skills:** `useEffect`, `setInterval`, cleanup functions

- Display a live-updating clock using `useEffect` + `setInterval`
- Build a stopwatch with Start / Pause / Reset
- Correctly clear the interval on unmount to avoid memory leaks

3. React To-Do App

Level: Beginner–Intermediate | **Core skills:** props, lists & keys, controlled forms, lifting state up

- Break the UI into `TodoForm`, `TodoList`, and `TodoItem` components
- Pass `add/toggle/delete` handlers down as props
- Persist todos to `localStorage` using `useEffect`

4. Weather App (Public API)

Level: Intermediate | **Core skills:** `useEffect` + `fetch`, loading/error states, env variables

- Fetch live weather data from a free public API by city name
- Show loading, error, and success states clearly in the UI
- Store the API key in a `.env` file, not hardcoded in the source

5. Expense Tracker

Level: Intermediate | **Core skills:** `useReducer`, derived state, form handling

- Add/remove income and expense entries using `useReducer` instead of `useState`
- Calculate and display running balance, total income, and total expenses
- Color-code entries (green for income, red for expenses)

6. Multi-Page Site with React Router

Level: Intermediate | **Core skills:** React Router, nested routes, route params, 404 handling

- Home / About / Blog List / Blog Detail pages with a shared navbar
- Use a dynamic route (e.g. `/blog/:id`) to show a single post
- Add a catch-all 404 page for unmatched routes

PHASE 2 — Node.js Core

Understand the runtime itself before hiding behind Express.

1. CLI File Organizer

Level: Beginner | **Core skills:** fs module, path module, synchronous vs async I/O

- Write a Node script that sorts files in a folder into subfolders by extension
- Use both fs.readdirSync and the promise-based fs/promises API and compare
- Log a summary of how many files were moved into each folder

2. Plain HTTP Server (No Framework)

Level: Beginner–Intermediate | **Core skills:** http module, manual routing, request/response objects

- Build a server using only Node's built-in http module
- Manually route GET /, GET /about, and a 404 fallback based on req.url
- Serve a small JSON API endpoint without any framework

3. Event Emitter Practice

Level: Intermediate | **Core skills:** EventEmitter, callbacks vs promises vs async/await

- Build a custom class that extends EventEmitter (e.g. an OrderProcessor)
- Emit and listen for custom events like 'orderPlaced' and 'orderShipped'
- Rewrite one callback-based function to return a Promise, then to use async/await

PHASE 3 — Express & REST APIs

Learn to design and build a real backend API.

1. Notes REST API (In-Memory)

Level: Beginner–Intermediate | **Core skills:** Express routing, CRUD, middleware, status codes

- Implement GET/POST/PUT/DELETE for /api/notes using an in-memory array
- Add a logging middleware that prints method + URL for every request
- Return correct HTTP status codes (200/201/404/400) for each case

2. Book Library API with Validation

Level: Intermediate | **Core skills:** express-validator, centralized error handling

- Validate request bodies (title, author, year) before creating/updating a book
- Return clear 400 responses listing exactly which fields failed validation
- Build a single centralized error-handling middleware instead of try/catch everywhere

3. File Upload API

Level: Intermediate | **Core skills:** multer, static file serving, MIME type checks

- Accept an image upload on POST /api/upload using multer
- Restrict uploads to image types and a max file size
- Serve uploaded files back via a static /uploads route

PHASE 4 — MongoDB & Mongoose

Model and query real data before wiring it to a frontend.

1. Movie Catalog CRUD

Level: Intermediate | **Core skills:** schema design, Mongoose models, CRUD operations

- Define a Movie schema (title, genre, rating, releaseYear) with Mongoose
- Implement full CRUD against a real MongoDB database (Atlas or local)
- Add query filtering, e.g. GET /api/movies?genre=Action&minRating;=7

2. Blog API with Relationships

Level: Intermediate–Advanced | **Core skills:** referencing vs embedding, populate(), schema relationships

- Model Users, Posts, and Comments as separate collections
- Reference Post -> User (author) and Comment -> Post, using ObjectId refs
- Use .populate() to return a post with its author's name and its comments

3. Sales Data Aggregation Practice

Level: Intermediate–Advanced | **Core skills:** aggregation pipeline: \$match, \$group, \$sort, \$lookup

- Seed a collection with sample order data
- Write an aggregation pipeline to get total sales per month
- Use \$lookup to join orders with a products collection and total revenue per product

PHASE 5 — Full-Stack CRUD (Connecting React + Express + Mongo)

The first projects where all three pieces of MERN talk to each other.

1. Contact Manager

Level: Intermediate | **Core skills:** axios/fetch from React, CORS, full CRUD wiring

- Build a React frontend that lists, adds, edits, and deletes contacts
- Wire it to your own Express + MongoDB API from Phase 3–4
- Correctly configure CORS and handle loading/error states in the UI

2. Kanban Task Board

Level: Intermediate–Advanced | **Core skills:** drag-and-drop, PATCH updates, optimistic UI

- Columns for To Do / In Progress / Done, with draggable task cards
- PATCH the task's status in MongoDB when it's dropped in a new column
- Update the UI optimistically, then roll back if the API call fails

PHASE 6 — Authentication & Authorization

Every real app needs this — learn it as its own focused unit.

1. Auth API (JWT + bcrypt)

Level: Intermediate–Advanced | **Core skills:** password hashing, JWT signing/verifying, protected routes

- Build /register and /login endpoints; hash passwords with bcrypt before saving
- Issue a signed JWT on login and verify it in a custom auth middleware
- Create one protected route (e.g. GET /api/profile) that requires a valid token

2. Full-Stack Login System

Level: Advanced | **Core skills:** httpOnly cookies vs localStorage tokens, protected React routes

- Build login/register forms in React that call your Auth API from project 1
- Store the JWT securely (httpOnly cookie recommended) and attach it to requests
- Create a PrivateRoute wrapper that redirects unauthenticated users to /login

3. Role-Based Access Control

Level: Advanced | **Core skills:** roles, middleware guards, conditional UI rendering

- Add a role field (user/admin) to your User model
- Write middleware that blocks non-admins from admin-only routes
- Hide/show admin UI sections in React based on the logged-in user's role

PHASE 7 — State Management at Scale

Move beyond prop-drilling once an app has real shared state.

1. Shopping Cart with Context API

Level: Intermediate | **Core skills:** Context API, useReducer for global state

- Build a product list + cart where cart state lives in a Context provider
- Support add, remove, and change-quantity actions via a reducer
- Show the cart item count in the navbar from anywhere in the app

2. Same Cart App, Rebuilt with Redux Toolkit

Level: Intermediate–Advanced | **Core skills:** Redux Toolkit slices, store setup, async thunks

- Recreate the cart from project 1 using Redux Toolkit instead of Context
- Add a cartSlice with add/remove/updateQuantity reducers
- Fetch the product list from your API using createAsyncThunk

PHASE 8 — Advanced React & Performance

Learn the patterns that matter once an app grows.

1. Infinite Scroll Feed

Level: Advanced | **Core skills:** pagination, IntersectionObserver, memoization

- Load a paginated list of items (e.g. posts) from your API
- Auto-load the next page when the user scrolls near the bottom
- Use React.memo / useMemo to avoid unnecessary re-renders of list items

2. Debounced Live Search

Level: Intermediate–Advanced | **Core skills:** custom hooks, useCallback, debouncing

- Build a search bar that queries your API as the user types
- Write a custom useDebounce hook instead of pulling in a library
- Cancel stale requests so an older response can't overwrite a newer one

3. Dark Mode with a Custom Hook

Level: Intermediate | **Core skills:** custom hooks, Context, localStorage persistence

- Build a useTheme hook backed by Context that toggles light/dark
- Persist the chosen theme in localStorage across reloads
- Apply theme classes/CSS variables across at least 3 different pages

PHASE 9 — Real-Time Features

Add live, push-based updates on top of your REST APIs.

1. Real-Time Chat App

Level: Advanced | **Core skills:** Socket.IO, rooms, broadcasting events

- Build a chat room where messages appear instantly for all connected users
- Support multiple named rooms using Socket.IO rooms
- Show a 'user is typing...' indicator using a custom socket event

2. Live Notification System

Level: Advanced | **Core skills:** event-driven UI updates, socket + REST combined

- When a user performs an action (e.g. new comment), emit a socket event to relevant users
- Show a live notification badge that updates without a page refresh
- Persist notifications in MongoDB so they still show up after a reload

PHASE 10 — Testing

Catch bugs before your users do.

1. API Testing Suite

Level: Intermediate–Advanced | **Core skills:** Jest, Supertest, integration testing

- Write unit tests for a few pure utility functions
- Write integration tests for your Express routes using Supertest
- Test both success cases and expected failure cases (bad input, missing auth)

2. React Component Testing

Level: Intermediate | **Core skills:** React Testing Library, user-event, mocking fetch

- Write tests that render components and assert on visible text/elements
- Simulate user interaction (typing, clicking) with @testing-library/user-event
- Mock an API call and test both the loading and loaded states

PHASE 11 — Deployment & Docker

Ship it — a project isn't done until someone else can open it.

1. Deploy a Full MERN App

Level: Intermediate–Advanced | **Core skills:** env vars, build scripts, production CORS, MongoDB Atlas

- Deploy the Express API to a host like Render or Railway
- Deploy the React build to Vercel or Netlify, pointing it at the live API URL
- Move all secrets into environment variables on both platforms

2. Dockerize a MERN App

Level: Advanced | **Core skills:** Dockerfile, docker-compose, multi-container apps

- Write a Dockerfile for the Express API and one for the React app
- Use docker-compose to run API + client + MongoDB together with one command
- Confirm the containerized app works identically to the local dev version

PHASE 12 — Capstone Projects

Full applications that combine every skill above. Build these to have real portfolio pieces.

1. Job Board Application

Level: Capstone | **Core skills:** auth + roles, CRUD, search/filter, pagination

- Two roles: employers post/manage jobs, seekers browse and apply
- Search and filter jobs by title, location, and category
- Paginate job listings and protect employer-only routes with your auth middleware

2. Social Media Mini-Clone

Level: Capstone | **Core skills:** posts, likes/comments, image upload, real-time notifications

- Users can create posts with an optional image, and like/comment on posts
- Real-time notification when someone likes or comments on your post (Socket.IO)
- Infinite-scrolling feed sorted by most recent

3. E-Commerce Store

Level: Capstone | **Core skills:** cart + checkout, admin dashboard, aggregation analytics, JWT auth

- Product catalog, cart, and a mock checkout flow (no real payment needed)
- Admin dashboard to add/edit products and view orders
- Use MongoDB aggregation to show an admin sales-by-month chart

Tip: don't rebuild every project from scratch each time — reuse your Phase 3–4 API code as the backend for the Phase 5+ projects wherever it fits. That reuse is itself a real MERN skill.